

# Reviewer Pack — Symmetric Group Orbit Count Distinguishes Read-Twice BPs

Entry 376a7658c1fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 05:31:34 UTC

## Symmetric Group Orbit Count Distinguishes Read-Twice BPs

**Entry ID:** 376a7658c1fe **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 05:31:34 UTC

### 1. Conjecture

**Field A** (mathematical branch): Invariant Theory **Field B** (complexity object): Read-Twice Branching Programs

#### Statement:

For a read-twice BP  $P$ , the number of orbits of its associated polynomial representation under the symmetric group  $S_n$  action is  $\Theta(\log \text{size}(P))$  for read-once BPs but  $\Omega(n)$  for read-twice BPs. The orbit count is strictly greater for read-twice BPs when  $\text{size}(P) \geq 2^{n/2}$ .

#### Rationale (proposer's reasoning):

Invariant theory provides a framework to measure symmetry in polynomial representations, which could expose structural differences between read-once and read-twice BPs. The symmetric group action on variables creates a natural invariant that may scale differently with BP complexity.

**Taxonomy category:** BP\_READTWICE (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** bd9398da59d2a3b3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier     | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|-------------|---------|------------|------------------|------------------|
| KARP_LIPTON | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=124, elapsed=245.3s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if sum(clause) != 0: # Ensure the clause is not trivial
                clauses.append(clause)
        return clauses

    def polynomial_representation(clauses):
        poly = {}
        for clause in clauses:
            monomial = 1
            for var in clause:
                if var > 0:
                    monomial *= (var + 1) / 2
                else:
                    monomial *= (-var + 1) / 2
            poly[tuple(sorted(clause))] = monomial
        return poly

    def symmetric_group_action(poly, n):
        actions = {}
        for i in range(n):
            action = [i]
            for j in range(1, n):
                action.append((action[-1] + 1) % n)
            actions[tuple(action)] = poly
        return actions

    def count_orbits(poly, actions):
        orbits = set()
        for action in actions.values():
            orbit = frozenset(action.keys())
            if orbit not in orbits:
```

```

        orbits.add(orbit)
    return len(orbits)

n = random.randint(5, 40)
clauses = generate_3cnf(n)
poly = polynomial_representation(clauses)
actions = symmetric_group_action(poly, n)
orbit_count = count_orbits(poly, actions)

if "read_once" in seed:
    expected_orbit_count = math.log(len(clauses))
else:
    expected_orbit_count = n / 2

return {
    "metric_name": "orbit_count",
    "metric_value": orbit_count,
    "instances_tested": 1,
    "conjecture_holds": orbit_count >= expected_orbit_count,
    "counterexample": "" if orbit_count >= expected_orbit_count else f"read_once={seed}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} stdev={sum((r['metric_value'] - support_fraction)**2 for r in results) / len(results)}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test timed out before producing results; insufficient data to evaluate support fraction or counterexamples. | next: Run test with extended timeout and higher resource allocation

## 11. Audit log (LLM calls)

**Total LLM calls:** 12

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 40268        |
| 2  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 138010       |
| 3  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 56013        |
| 4  | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 99206        |
| 5  | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 31817        |
| 6  | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 27292        |
| 7  | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20375        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 40881        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9351         |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 6342         |
| 11 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8708         |
| 12 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 136342       |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 614607 ms total latency. Provider mix: {'ollama\_remote': 12}

*(full prompt+response transcripts available in research/audit/376a7658c1fe.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/376a7658c1fe.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/376a7658c1fe.tar.gz (if generated)